

VISUAL BASIC .NET

Unit-4: Arrays, Collections, File I/O, Database Access, Controls & Debugging

Comprehensive Study Notes

Himachal Pradesh University – BCA Semester IV

4.1 Arrays

What are Arrays?

An **array** is a data structure that stores a **fixed-size** sequential collection of elements of the **same type**. Arrays help manage multiple values in a single variable.

Why Use Arrays?

Reason	Explanation
Organization	Group related data together
Efficiency	Access elements using index
Memory Management	Contiguous memory allocation
Iteration	Easy to loop through elements
Code Reusability	Write once, use for all elements

4.1.1 One-Dimensional Arrays

A **one-dimensional array** stores elements in a **single row** (linear sequence). Each element is accessed using an **index** starting from 0.

Declaration

```
Dim arrayName(size) As DataType
```

Examples

```
' Declaration with size
Dim numbers(4) As Integer      ' 5 elements (index 0-4)
Dim marks(9) As Double        ' 10 elements

' Declaration with initialization
Dim names() As String = {"John", "Jane", "Bob"}
Dim scores As Integer() = {85, 90, 78, 92, 88}

' Array Representation:
' Index:  0   1   2   3   4
'          +---+---+---+---+---+
'          |85 |90 |78 |92 |88 |
'          +---+---+---+---+---+

' Accessing elements
Dim first As Integer = scores(0) ' 85
Dim third As Integer = scores(2) ' 78
scores(4) = 95                    ' Change last element
```

4.1.2 Assign Values to Array Elements

```
' Method 1: Initialize at declaration
Dim arr1() As Integer = {10, 20, 30, 40, 50}

' Method 2: Using index
Dim arr2(4) As Integer
arr2(0) = 10
arr2(1) = 20
arr2(2) = 30
arr2(3) = 40
arr2(4) = 50

' Method 3: Using loop
Dim arr3(9) As Integer
For i As Integer = 0 To 9
    arr3(i) = (i + 1) * 10
Next

' Method 4: User input
Dim arr4(4) As Integer
For i As Integer = 0 To 4
    arr4(i) = CInt(InputBox("Enter value for index " & i))
Next
```

4.1.3 Use For Loops to Work with Arrays

```
Public Class Form1
    Private Sub ArrayForLoopExamples()
        Dim scores() As Integer = {85, 90, 78, 92, 88}

        ' Display all elements
        For i As Integer = 0 To scores.Length - 1
            MessageBox.Show("Score " & i & ": " & scores(i))
        Next

        ' Calculate sum and average
        Dim sum As Integer = 0
        For i As Integer = 0 To scores.Length - 1
            sum += scores(i)
        Next
        Dim avg As Double = sum / scores.Length

        ' Find maximum
        Dim max As Integer = scores(0)
        For i As Integer = 1 To scores.Length - 1
            If scores(i) > max Then max = scores(i)
        Next
    End Sub
End Class
```

4.1.4 Use For Each Loops to Work with Arrays

```
Public Class Form1
    Private Sub ArrayForEachExamples()
        Dim fruits() As String = {"Apple", "Banana", "Cherry", "Date"}

        ' Display all elements using For Each
        Dim output As String = ""
        For Each fruit As String In fruits
            output &= fruit & vbCrLf
        Next
        MessageBox.Show(output)

        ' For Each cannot modify elements directly
        ' Use For loop when modification is needed
    End Sub
End Class
```

For vs For Each – Comparison

Aspect	For Loop	For Each Loop
Access	By index	Direct element
Modification	Can modify elements	Cannot modify directly
Index needed	Yes	No
Performance	Slightly faster	Slightly slower
Readability	More verbose	More readable
Use when	Need index	Need only values

4.1.5 Work with Rectangular Arrays (2D)

A **rectangular array** has two dimensions – rows and columns. It is like a **table**.

```
Dim matrix(rows, cols) As DataType
```

Declaration and Usage

```
' 2D array: 3 rows, 4 columns
Dim matrix(2, 3) As Integer

' Assign values
matrix(0, 0) = 1: matrix(0, 1) = 2: matrix(0, 2) = 3: matrix(0, 3) = 4
matrix(1, 0) = 5: matrix(1, 1) = 6: matrix(1, 2) = 7: matrix(1, 3) = 8
matrix(2, 0) = 9: matrix(2, 1) = 10: matrix(2, 2) = 11: matrix(2, 3) = 12

' Initialize at declaration
Dim grid(,) As Integer = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
}

' Loop through 2D array
For row As Integer = 0 To grid.GetLength(0) - 1
    Dim rowText As String = ""
    For col As Integer = 0 To grid.GetLength(1) - 1
        rowText &= grid(row, col) & vbTab
    Next
    MessageBox.Show("Row " & row & ": " & rowText)
Next
```

4.1.6 Create a Jagged Array

A **jagged array** is an **array of arrays** where each row can have a **different length**.

```
Dim jaggedArray()() As DataType
```

Example

```
' Declare jagged array (3 rows)
Dim jagged()() As Integer = New Integer(2)() {}

' Each row has different length
jagged(0) = New Integer() {1, 2, 3}           ' 3 elements
jagged(1) = New Integer() {4, 5}               ' 2 elements
jagged(2) = New Integer() {6, 7, 8, 9}         ' 4 elements

' Access elements
Dim val As Integer = jagged(1)(0) ' 4

' Loop through jagged array
For i As Integer = 0 To jagged.Length - 1
    Dim rowText As String = ""
    For j As Integer = 0 To jagged(i).Length - 1
        rowText &= jagged(i)(j) & " "
    Next
    MessageBox.Show("Row " & i & ": " & rowText)
Next
```

Rectangular vs Jagged Arrays

Aspect	Rectangular Array	Jagged Array
Structure	Matrix (rows x cols)	Array of arrays
Column Length	Fixed (same for all rows)	Variable (different per row)
Memory	Contiguous	Non-contiguous
Declaration	Dim arr(rows, cols)	Dim arr()()
Access	arr(row, col)	arr(row)(col)

4.1.7 Use the Array Class

The **Array class** provides static methods and properties for working with arrays.

Properties

Property	Description
Length	Total number of elements
Rank	Number of dimensions
IsFixedSize	True if array has fixed size
IsReadOnly	True if array is read-only

Methods

Method	Description
Array.Clear()	Clears all elements
Array.Copy()	Copies elements to another array
Array.IndexOf()	Finds first occurrence
Array.LastIndexOf()	Finds last occurrence
Array.Reverse()	Reverses the order
Array.Sort()	Sorts the array
Array.BinarySearch()	Searches sorted array
Array.Find()	Finds element matching condition
Array.FindAll()	Finds all matching elements
Array.Exists()	Checks if element exists
Array.Resize()	Changes array size
Array.TrueForAll()	Checks all elements satisfy condition

Examples

```
Private Sub ArrayClassExamples()  
    Dim numbers() As Integer = {5, 2, 8, 1, 9, 3, 7}  
  
    ' Sort  
    Array.Sort(numbers)           ' {1, 2, 3, 5, 7, 8, 9}  
  
    ' Reverse  
    Array.Reverse(numbers)        ' {9, 8, 7, 5, 3, 2, 1}  
  
    ' Find  
    Dim found As Integer = Array.Find(numbers, Function(x) x > 5)  
    ' Returns first element > 5  
  
    ' IndexOf  
    Dim idx As Integer = Array.IndexOf(numbers, 7)  
  
    ' Clear (set to default)  
    Array.Clear(numbers, 0, 3)    ' {0, 0, 0, 5, 3, 2, 1}  
  
    ' Resize  
    Array.Resize(numbers, 10)     ' Now has 10 elements  
  
    ' Exists  
    Dim hasEven As Boolean = Array.Exists(numbers, Function(x) x Mod 2 = 0)  
  
    ' CopyTo  
    Dim copy(9) As Integer  
    numbers.CopyTo(copy, 0)  
  
    ' Clone  
    Dim cloned() As Integer = CType(numbers.Clone(), Integer())  
  
    ' GetLength for multidimensional  
    Dim grid(,) As Integer = {{1, 2}, {3, 4}}  
    Dim rows As Integer = grid.GetLength(0)  
    Dim cols As Integer = grid.GetLength(1)  
End Sub
```

4.1.8 Refer to and Copy Arrays

```
Private Sub ArrayReferenceAndCopy()  
    Dim original() As Integer = {1, 2, 3, 4, 5}  
  
    ' Shallow copy (same reference)  
    Dim sameRef() As Integer = original  
    sameRef(0) = 99 ' Also changes original(0) to 99  
  
    ' Deep copy using Clone  
    Dim deepCopy() As Integer = CType(original.Clone(), Integer())  
    deepCopy(0) = 50 ' Does NOT affect original  
  
    ' Deep copy using CopyTo  
    Dim copyTo(4) As Integer  
    original.CopyTo(copyTo, 0)  
  
    ' Deep copy using Array.Copy  
    Dim dest(4) As Integer  
    Array.Copy(original, dest, original.Length)  
End Sub
```

4.1.9 Code Procedures That Work with Arrays

```
Public Class Form1
    ' Procedure accepting array
    Private Function CalculateAverage(ByVal arr() As Double) As Double
        Dim sum As Double = 0
        For Each val As Double In arr
            sum += val
        Next
        Return sum / arr.Length
    End Function

    ' Function returning array
    Private Function GenerateNumbers(ByVal count As Integer) As Integer()
        Dim result(count - 1) As Integer
        For i As Integer = 0 To count - 1
            result(i) = (i + 1) * 10
        Next
        Return result
    End Function

    ' Procedure modifying array (ByRef implied for array)
    Private Sub DoubleAllValues(ByVal arr() As Integer)
        For i As Integer = 0 To arr.Length - 1
            arr(i) = arr(i) * 2
        Next
    End Sub

    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles
        Button1.Click
            Dim scores() As Double = {85.5, 90.0, 78.5}
            Dim avg As Double = CalculateAverage(scores)

            Dim nums() As Integer = GenerateNumbers(5)
            DoubleAllValues(nums)
        End Sub
    End Class
```

4.1.10 Work with Collections

Collections Overview

Collection	Description	Type-Safe
List(Of T)	Dynamic array, index-based	Yes
ArrayList	Dynamic array (non-generic)	No
Dictionary(Of K,V)	Key-value pairs	Yes
Stack(Of T)	LIFO (Last-In-First-Out)	Yes
Queue(Of T)	FIFO (First-In-First-Out)	Yes
SortedList	Sorted key-value pairs	No
Hashtable	Key-value pairs (non-generic)	No

List(Of T)

```
Imports System.Collections.Generic

Private Sub ListExamples()
    Dim students As New List(Of String)()

    ' Add items
    students.Add("Alice")
    students.Add("Bob")
    students.Add("Charlie")
    students.Insert(1, "David") ' Insert at index 1

    ' Remove
    students.Remove("Bob")      ' Remove by value
    students.RemoveAt(0)        ' Remove by index

    ' Query
    Dim exists As Boolean = students.Contains("Charlie")
    Dim count As Integer = students.Count
    Dim first As String = students(0) ' Index access

    ' Sort
    students.Sort()
```

```
' Find
Dim found As String = students.Find(Function(s) s.StartsWith("C"))

' Clear
students.Clear()

End Sub
```

Dictionary(Of K, V)

```
Private Sub DictionaryExamples()
    Dim employees As New Dictionary(Of Integer, String)()

    ' Add
    employees.Add(101, "Alice")
    employees.Add(102, "Bob")
    employees.Add(103, "Charlie")

    ' Access
    Dim name As String = employees(101)      ' "Alice"

    ' Update
    employees(101) = "Alicia"

    ' Check existence
    If employees.ContainsKey(104) Then
        Dim emp As String = employees(104)
    End If

    ' Keys and Values
    Dim allKeys As Dictionary(Of Integer, String).KeyCollection = employees.Keys
    Dim allValues As Dictionary(Of Integer, String).ValueCollection =
employees.Values

    ' Iterate
    For Each kvp As KeyValuePair(Of Integer, String) In employees
        MessageBox.Show(kvp.Key & ": " & kvp.Value)
    Next

    ' Remove
    employees.Remove(102)

    ' Count
    Dim empCount As Integer = employees.Count

End Sub
```

Stack(Of T) – LIFO

```
Private Sub StackExamples()  
    Dim undoStack As New Stack(Of String)()  
  
    ' Push (add to top)  
    undoStack.Push("Action 1")  
    undoStack.Push("Action 2")  
    undoStack.Push("Action 3")  
  
    ' Peek (view top without removing)  
    Dim topAction As String = undoStack.Peek() ' "Action 3"  
  
    ' Pop (remove and return top)  
    Dim lastAction As String = undoStack.Pop() ' "Action 3"  
  
    ' Contains  
    Dim hasAction As Boolean = undoStack.Contains("Action 1")  
  
    ' Count  
    Dim stackCount As Integer = undoStack.Count  
End Sub
```

Queue(Of T) – FIFO

```
Private Sub QueueExamples()  
    Dim printQueue As New Queue(Of String)()  
  
    ' Enqueue (add to back)  
    printQueue.Enqueue("Doc1.pdf")  
    printQueue.Enqueue("Doc2.pdf")  
    printQueue.Enqueue("Doc3.pdf")  
  
    ' Peek (view front without removing)  
    Dim nextDoc As String = printQueue.Peek() ' "Doc1.pdf"  
  
    ' Dequeue (remove and return front)  
    Dim printedDoc As String = printQueue.Dequeue() ' "Doc1.pdf"  
  
    ' Contains  
    Dim hasDoc As Boolean = printQueue.Contains("Doc2.pdf")  
  
    ' Count  
    Dim queueCount As Integer = printQueue.Count
```

```
' Iterate
For Each doc As String In printQueue
    MessageBox.Show(doc)
Next
End Sub
```

ArrayList (Non-Generic)

```
Imports System.Collections

Private Sub ArrayListExamples()
    Dim list As New ArrayList()

    ' Can hold different types
    list.Add(10)
    list.Add("Hello")
    list.Add(3.14)
    list.Add(True)

    ' Access (needs casting)
    Dim num As Integer = CInt(list(0))
    Dim text As String = CStr(list(1))

    ' Sort (all elements must be comparable)
    ' Remove, Insert, Contains all work similar to List(Of T)

    ' Count
    Dim itemCount As Integer = list.Count
End Sub
```

Hashtable (Non-Generic)

```
Imports System.Collections

Private Sub HashtableExamples()
    Dim ht As New Hashtable()

    ht.Add("Name", "Alice")
    ht.Add("Age", 25)
    ht.Add("City", "New York")

    ' Access
```

```
Dim name As String = CStr(ht("Name"))

' ContainsKey
If ht.ContainsKey("Age") Then
    Dim age As Integer = CInt(ht("Age"))
End If

' Keys and Values
For Each key As String In ht.Keys
    MessageBox.Show(key & ": " & ht(key).ToString())
Next

ht.Remove("City")
End Sub
```

SortedList

```
Imports System.Collections

Private Sub SortedListExamples()
    Dim sl As New SortedList()

    ' Automatically sorted by key
    sl.Add(103, "Charlie")
    sl.Add(101, "Alice")
    sl.Add(102, "Bob")

    ' Now stored: 101-Alice, 102-Bob, 103-Charlie

    For Each key As Integer In sl.Keys
        MessageBox.Show(key & ": " & sl(key).ToString())
    Next
End Sub
```

4.2 Dates, Strings, Formatting & Validation

4.2.1 Create a DateTime Value

```
Private Sub CreateDateTimeExamples()
    ' Specific date/time
    Dim dt1 As New DateTime(2026, 7, 11)           ' July 11, 2026
    Dim dt2 As New DateTime(2026, 7, 11, 14, 30, 0) ' July 11, 2026, 2:30 PM

    ' Using date literals
    Dim dt3 As Date = #7/11/2026#

    ' Parse from string
    Dim dt4 As DateTime = DateTime.Parse("2026-07-11")
    Dim dt5 As DateTime = DateTime.ParseExact("11/07/2026", "dd/MM/yyyy", Nothing)

    ' From components
    Dim dt6 As DateTime = New DateTime(2026, 12, 25, 10, 30, 0)
End Sub
```

4.2.2 Get the Current Date & Time

```
Private Sub CurrentDateTime()
    Dim now As DateTime = DateTime.Now           ' Current date and time
    Dim today As DateTime = DateTime.Today       ' Current date (time = 00:00:00)
    Dim utcNow As DateTime = DateTime.UtcNow    ' UTC time

    ' Components
    Dim year As Integer = now.Year
    Dim month As Integer = now.Month
    Dim day As Integer = now.Day
    Dim hour As Integer = now.Hour
    Dim minute As Integer = now.Minute
    Dim second As Integer = now.Second
    Dim dayOfWeek As DayOfWeek = now.DayOfWeek
End Sub
```

4.2.3 Format DateTime Values

Specifier	Description	Example
-----------	-------------	---------

d	Short date	7/11/2026
D	Long date	Saturday, July 11, 2026
t	Short time	2:30 PM
T	Long time	2:30:00 PM
f	Full date/time (short)	Saturday, July 11, 2026 2:30 PM
F	Full date/time (long)	Saturday, July 11, 2026 2:30:00 PM
g	General (short)	7/11/2026 2:30 PM
G	General (long)	7/11/2026 2:30:00 PM
M / m	Month/day	July 11
Y / y	Year/month	July 2026
dd	Day (01-31)	11
ddd	Short day name	Sat
dddd	Full day name	Saturday
MM	Month (01-12)	07
MMM	Short month name	Jul
MMMM	Full month name	July
yy	Two-digit year	26
yyyy	Four-digit year	2026
hh	12-hour hour	02
HH	24-hour hour	14
mm	Minutes	30
ss	Seconds	00
tt	AM/PM	PM

```

Private Sub FormatDateTimeExamples()
    Dim dt As DateTime = Now

    Dim shortDate As String = dt.ToString("d")      ' 7/11/2026

```

```
Dim longDate As String = dt.ToString("D")      ' Saturday, July 11, 2026
Dim custom As String = dt.ToString("dd-MM-yyyy HH:mm:ss")
Dim monthYear As String = dt.ToString("MMMM yyyy")
Dim dayName As String = dt.ToString("dddd")
End Sub
```

4.2.4 Perform Operations on Dates and Times

```
Private Sub DateOperations()
    Dim startDate As DateTime = #1/1/2026#
    Dim endDate As DateTime = #7/11/2026#

    ' Difference
    Dim diff As TimeSpan = endDate - startDate
    Dim daysDiff As Integer = diff.Days

    ' Add days, months, years
    Dim nextWeek As DateTime = startDate.AddDays(7)
    Dim nextMonth As DateTime = startDate.AddMonths(1)
    Dim nextYear As DateTime = startDate.AddYears(1)

    ' Add hours, minutes, seconds
    Dim later As DateTime = startDate.AddHours(5).AddMinutes(30)

    ' DateDiff function
    Dim monthsDiff As Long = DateDiff(DateInterval.Month, startDate, endDate)

    ' DateAdd function
    Dim newDate As DateTime = DateAdd(DateInterval.Day, 45, startDate)

    ' Compare
    Dim result As Integer = DateTime.Compare(startDate, endDate)
    ' -1: earlier, 0: same, 1: later
End Sub
```

4.2.5 Work with Strings

```
Private Sub AdvancedStringMethods()
    Dim text As String = " Visual Basic .NET Programming "
```

```
Dim rightTrim As String = text.TrimEnd()

' Case conversion
Dim upper As String = trimmed.ToUpper()
Dim lower As String = trimmed.ToLower()

' Substring
Dim subStr As String = trimmed.Substring(7, 5)    ' "Basic"

' Split and Join
Dim csv As String = "apple,banana,orange,grape"
Dim parts() As String = csv.Split(",")
Dim joined As String = String.Join(" | ", parts)

' Replace
Dim replaced As String = trimmed.Replace("VB", "Visual Basic")

' Contains, StartsWith, EndsWith
Dim hasBasic As Boolean = trimmed.Contains("Basic")
Dim startsWith As Boolean = trimmed.StartsWith("Visual")
Dim endsWith As Boolean = trimmed.EndsWith("ing")

' IndexOf, LastIndexOf
Dim firstPos As Integer = trimmed.IndexOf("a")
Dim lastPos As Integer = trimmed.LastIndexOf("a")

' Length
Dim len As Integer = trimmed.Length

' PadLeft, PadRight
Dim padded As String = "5".PadLeft(3, "0")    ' "005"

' Remove, Insert
Dim without As String = trimmed.Remove(7, 6)
Dim withInsert As String = trimmed.Insert(7, "Awesome ")
End Sub
```

4.2.6 Procedures for Validating User Entries

```
Public Class Form1
    Private Function IsValidName(ByVal name As String) As Boolean
        Return Not String.IsNullOrEmpty(name) AndAlso name.Length >= 2
    End Function
End Class
```

```

    Private Function IsValidInteger(ByVal input As String, ByRef value As Integer)
As Boolean
        Return Integer.TryParse(input, value)
    End Function

    Private Function IsValidDouble(ByVal input As String, ByRef value As Double) As
Boolean
        Return Double.TryParse(input, value)
    End Function

    Private Function IsValidEmail(ByVal email As String) As Boolean
        Return email.Contains("@") AndAlso email.Contains(".") AndAlso email.Length
> 5
    End Function

    Private Sub btnValidate_Click(sender As Object, e As EventArgs) Handles
btnValidate.Click
        Dim age As Integer
        If Not IsValidName(txtName.Text) Then
            MessageBox.Show("Enter a valid name (min 2 characters).", "Error")
        ElseIf Not IsValidInteger(txtAge.Text, age) OrElse age < 0 OrElse age > 150
Then
            MessageBox.Show("Enter a valid age (0-150).", "Error")
        ElseIf Not IsValidEmail(txtEmail.Text) Then
            MessageBox.Show("Enter a valid email address.", "Error")
        Else
            MessageBox.Show("All inputs are valid!", "Success")
        End If
    End Sub
End Class

```

4.2.7 Format Numbers, Dates, and Times

```

Private Sub FormatNumberExamples()
    Dim price As Double = 12345.6789
    Dim pct As Double = 0.1567

    ' Number formats
    Dim n1 As String = price.ToString("N2")           ' 12,345.68
    Dim n2 As String = price.ToString("C")           ' $12,345.68 (currency)
    Dim n3 As String = price.ToString("F3")           ' 12345.679 (fixed point)
    Dim n4 As String = price.ToString("#,##0.00")     ' 12,345.68

    ' Percentage

```

```
Dim p1 As String = pct.ToString("P1")           ' 15.7%

' Custom formats
Dim f1 As String = Format(price, "$#,##0.00")
Dim f2 As String = FormatCurrency(price)
Dim f3 As String = FormatNumber(price, 2)
Dim f4 As String = FormatPercent(pct, 2)

' Date formats
Dim dt As DateTime = Now
Dim d1 As String = Format(dt, "dd/MM/yyyy")
Dim d2 As String = Format(dt, "dddd, MMMM dd, yyyy")
Dim d3 As String = FormatDateTime(dt, DateFormat.LongDate)
End Sub
```

4.3 Common Controls

4.3.1 ComboBox, ListBox, CheckBox, RadioButton, GroupBox

ComboBox

```
Private Sub ComboBoxExamples()  
    ' Add items  
    ComboBox1.Items.Add("Computer Science")  
    ComboBox1.Items.Add("Mathematics")  
    ComboBox1.Items.Add("Physics")  
    ComboBox1.Items.AddRange({"Chemistry", "Biology"})  
  
    ' Select  
    ComboBox1.SelectedIndex = 0 ' Select first  
    ComboBox1.Text = "Select a course..."  
  
    ' Get selected  
    Dim selectedItem As String = ComboBox1.SelectedItem.ToString()  
    Dim selectedIndex As Integer = ComboBox1.SelectedIndex  
  
    ' Clear and count  
    Dim itemCount As Integer = ComboBox1.Items.Count  
    ' ComboBox1.Items.Clear()  
End Sub  
  
Private Sub ComboBox1_SelectedIndexChanged(sender As Object, e As EventArgs) _  
    Handles ComboBox1.SelectedIndexChanged  
    MessageBox.Show("Selected: " & ComboBox1.SelectedItem.ToString())  
End Sub
```

ListBox

```
Private Sub ListBoxExamples()  
    ' Add items  
    ListBox1.Items.Add("Item 1")  
    ListBox1.Items.Add("Item 2")  
    ListBox1.Items.AddRange({"Item A", "Item B", "Item C"})  
  
    ' Selection mode  
    ListBox1.SelectionMode = SelectionMode.One ' Single selection
```

```

' ListBox1.SelectionMode = SelectionMode.MultiSimple
' ListBox1.SelectionMode = SelectionMode.MultiExtended

' Get selected item(s)
If ListBox1.SelectedIndex >= 0 Then
    Dim selected As String = ListBox1.SelectedItem.ToString()
End If

' For multi-select
For Each item As String In ListBox1.SelectedItems
    MessageBox.Show(item)
Next

' Remove
ListBox1.Items.RemoveAt(0)
ListBox1.Items.Remove("Item A")

' Sort
ListBox1.Sorted = True
End Sub

```

CheckBox

```

Private Sub CheckBox1_CheckedChanged(sender As Object, e As EventArgs) _
    Handles CheckBox1.CheckedChanged, CheckBox2.CheckedChanged,
    CheckBox3.CheckedChanged
    Dim selected As String = "Selected: "
    If CheckBox1.Checked Then selected &= "Option 1 "
    If CheckBox2.Checked Then selected &= "Option 2 "
    If CheckBox3.Checked Then selected &= "Option 3 "
    Label1.Text = selected
End Sub

```

RadioButton (with GroupBox)

```

' RadioButtons inside a GroupBox are mutually exclusive
' Different GroupBoxes create separate groups

Private Sub RadioButton_CheckedChanged(sender As Object, e As EventArgs) _
    Handles RadioButton1.CheckedChanged, RadioButton2.CheckedChanged, _
    RadioButton3.CheckedChanged
    Dim rb As RadioButton = CType(sender, RadioButton)
    If rb.Checked Then

```

```
        MessageBox.Show("You selected: " & rb.Text)
    End If
End Sub
```

Complete Controls Example

```
Public Class Form1
    Private Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles
        btnSubmit.Click
            Dim result As String = ""

            ' TextBox
            result &= "Name: " & txtName.Text & vbCrLf

            ' ComboBox
            If cboCourse.SelectedIndex >= 0 Then
                result &= "Course: " & cboCourse.SelectedItem.ToString() & vbCrLf
            End If

            ' CheckBoxes (hobbies)
            result &= "Hobbies: "
            If chkReading.Checked Then result &= "Reading "
            If chkMusic.Checked Then result &= "Music "
            If chkSports.Checked Then result &= "Sports "
            result &= vbCrLf

            ' RadioButtons (gender)
            If rbMale.Checked Then result &= "Gender: Male"
            If rbFemale.Checked Then result &= "Gender: Female"

            MessageBox.Show(result, "Registration Summary")
        End Sub
    End Class
```

4.3.2 Use Tab Order View to Set the Tab Order

Tab order determines the sequence in which controls receive focus when the user presses the Tab key.

To set tab order:

1. Open the form in Design view
2. Click **View > Tab Order** (or click the Tab Order button on the Layout toolbar)

3. Click each control in the desired order (numbers will appear on each control)
4. Press Esc to exit Tab Order mode

Setting tab order programmatically:

```
Private Sub SetTabOrder()  
    txtName.TabIndex = 0  
    cboCourse.TabIndex = 1  
    chkReading.TabIndex = 2  
    rbMale.TabIndex = 3  
    btnSubmit.TabIndex = 4  
    btnCancel.TabIndex = 5  
End Sub
```

4.4 Multi-Form Projects

4.4.1 Add a Form to a Project

Method 1 – Using Visual Studio:

1. Right-click project in Solution Explorer
2. Select **Add > Windows Form**
3. Enter form name (e.g., Form2.vb)
4. Click Add

Method 2 – Programmatically:

```
' Create and show form
Dim frm As New Form2()
frm.Show()           ' Modeless (can switch between forms)
' frm.ShowDialog() ' Modal (must close before continuing)
```

4.4.2 Rename a Form

Methods:

1. In Solution Explorer: Right-click > Rename
2. In Properties Window: Change the Name property
3. In Code: Change the class name

```
' Renamed from Form1 to MainForm
Public Class MainForm
    ' ...
End Class
```

4.4.3 Change the Startup Form

Method 1 – Visual Studio:

1. Right-click project > Properties
2. Application tab > Startup form dropdown
3. Select the desired form

Method 2 – Using Sub Main:

```
Module Module1
    Public Sub Main()
        Application.Run(New LoginForm())
    End Sub
End Module
```

4.4.4 Display a Form as a Dialog Box

```
' Form1 code
Private Sub btnOpenDialog_Click(sender As Object, e As EventArgs) Handles
btnOpenDialog.Click
    Dim dlg As New DialogForm()
    Dim result As DialogResult = dlg.ShowDialog()

    If result = DialogResult.OK Then
        MessageBox.Show("Dialog returned OK")
    ElseIf result = DialogResult.Cancel Then
        MessageBox.Show("Dialog was cancelled")
    End If
    dlg.Dispose()
End Sub

' DialogForm code
Public Class DialogForm
    Private Sub btnOK_Click(sender As Object, e As EventArgs) Handles btnOK.Click
        Me.DialogResult = DialogResult.OK
        Me.Close()
    End Sub

    Private Sub btnCancel_Click(sender As Object, e As EventArgs) Handles
btnCancel.Click
        Me.DialogResult = DialogResult.Cancel
        Me.Close()
    End Sub
End Class
```

4.4.5 Pass Data Between Forms

Method 1 – Using Public Properties/Variables

```
' Form1 (Main Form)
Public Class Form1
```

```

        Private Sub btnOpen_Click(sender As Object, e As EventArgs) Handles
btnOpen.Click
            Dim frm As New Form2()
            frm.UserName = txtName.Text    ' Set property
            frm.ShowDialog()
            MessageBox.Show("Returned: " & frm.Result)    ' Read property
        End Sub
    End Class

' Form2
Public Class Form2
    Public Property UserName As String
    Public Property Result As String

    Private Sub Form2_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        lblWelcome.Text = "Welcome, " & UserName
    End Sub

    Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles
btnSave.Click
        Result = txtData.Text
        Me.DialogResult = DialogResult.OK
        Me.Close()
    End Sub
End Class

```

Method 2 – Using Constructor

```

' Form1
Private Sub btnOpen_Click(sender As Object, e As EventArgs) Handles btnOpen.Click
    Dim frm As New Form2(txtName.Text, txtAge.Text)
    frm.ShowDialog()
End Sub

' Form2
Public Class Form2
    Private _name As String
    Private _age As String

    Public Sub New(ByVal name As String, ByVal age As String)
        InitializeComponent()
        _name = name
        _age = age
    End Sub

```

```
Private Sub Form2_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    lblInfo.Text = "Name: " & _name & ", Age: " & _age
End Sub
End Class
```

4.4.6 MessageBox – Display and Get User Response

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
    Dim result As DialogResult = MessageBox.Show(
        "Do you want to save changes?",
        "Confirm Save",
        MessageBoxButtons.YesNoCancel,
        MessageBoxIcon.Question,
        MessageBoxDefaultButton.Button1)

    Select Case result
        Case DialogResult.Yes
            MessageBox.Show("Saving...", "Info",
                MessageBoxButtons.OK, MessageBoxIcon.Information)
        Case DialogResult.No
            MessageBox.Show("Discarding...", "Info",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Case DialogResult.Cancel
            ' Do nothing
    End Select
End Sub
```

4.4.7 Use the FormClosing Event

```
Private Sub Form1_FormClosing(sender As Object, e As FormClosingEventArgs) _
    Handles MyBase.FormClosing
    Dim result As DialogResult = MessageBox.Show(
        "Are you sure you want to close?",
        "Confirm Exit",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question)

    If result = DialogResult.No Then
        e.Cancel = True ' Prevents form from closing
    End If
End Sub
```

```
' FormClosingEventArgs properties  
' e.Cancel - Set to True to cancel closing  
' e.CloseReason - Enum: UserClosing, WindowsShutDown, TaskManagerClosing, etc.
```

4.5 Debugging

4.5.1 Debugging Options and Break Mode

Debugging Windows

Window	Description
Locals Window	Shows all local variables with their current values
Autos Window	Shows variables used in current and previous lines
Watch Window	Monitor specific expressions
Call Stack Window	Shows method call hierarchy
Immediate Window	Execute code on-the-fly during debugging
Output Window	Shows debug output messages

Break Mode

- Execution is **paused**
- Current line is highlighted in **yellow**
- Can examine variable values (hover mouse)
- Can edit code (Edit & Continue)
- Can execute statements in Immediate Window

Conditional Compilation

```
Private Sub ConditionalCompilation()  
#If DEBUG Then  
    MessageBox.Show("Debug mode - " & DateTime.Now.ToString())  
    Console.WriteLine("Debug output")  
#End If  
End Sub  
  
' Debug output  
Private Sub DebugOutput()  
    Debug.WriteLine("Entering method CalculateTotal")  
    Debug.WriteLine("Value: " & someVariable)
```

```
Trace.WriteLine("Trace output - always included")
End Sub
```

4.5.2 Edit and Continue Feature

Edit and Continue allows you to edit code while the application is in break mode. Changes are applied immediately when you continue execution.

- Edit code in break mode (yellow highlighted line)
- Press F5 or Continue to apply changes
- Limitations: Cannot change active statements, method signatures, or certain structural elements

4.5.3 Breakpoints

Breakpoint Type	Description
Simple Breakpoint	Pauses at line (F9)
Conditional Breakpoint	Pauses when condition is true
Tracepoint	Logs message without pausing
Function Breakpoint	Pauses when function is called

```
Private Sub BreakpointExamples()
    For i As Integer = 0 To 100
        ' Set conditional breakpoint: i = 50
        ' Condition: i == 50
        Debug.WriteLine(i)
    Next
End Sub
```

4.5.4 Debugging Windows

Locals Window

```
Private Sub LocalsWindowExample()
    Dim name As String = "Alice"
    Dim age As Integer = 25
    Dim marks() As Integer = {85, 90, 78}
    Dim total As Double = 0
```

```
For Each m As Integer In marks
    total += m
Next

' In break mode, Locals window shows:
' name = "Alice" (String)
' age = 25 (Integer)
' marks = {Length=3} (Integer())
' total = 253.0 (Double)
End Sub
```

Watch Window

```
Private Sub WatchWindowExample()
    Dim a As Integer = 10
    Dim b As Integer = 20

    ' Add watches:
    ' a          -> 10
    ' b          -> 20
    ' a + b      -> 30
    ' a * b      -> 200
    ' String.Format("{0} + {1} = {2}", a, b, a + b) -> "10 + 20 = 30"

    a = 50 ' Watches update automatically
End Sub
```

Call Stack Window

```
Private Sub MainFunction()
    CalculateTotal()
End Sub

Private Sub CalculateTotal()
    ValidateInput() ' Breakpoint here shows:
    ' Call Stack:
    '   ValidateInput()
    '   CalculateTotal()
    '   MainFunction()
```

```
End Sub

Private Sub ValidateInput()
    Debug.WriteLine("Validating...")
End Sub
```

Output Window

```
Imports System.Diagnostics

Private Sub OutputWindowExample()
    Debug.WriteLine("Debug message - only in Debug build")
    Trace.WriteLine("Trace message - Debug and Release builds")
    Console.WriteLine("Console output")

    ' IndexOutOfRangeException example
    Try
        Dim arr(4) As Integer
        arr(10) = 5
    Catch ex As Exception
        Debug.WriteLine("Exception: " & ex.Message)
        Debug.WriteLine("Stack Trace: " & ex.StackTrace)
    End Try
End Sub
```

Immediate Window

```
' During break mode, type in Immediate Window:
' ? variableName      -> displays value
' ? a + b             -> evaluates expression
' variableName = newValue -> changes value
' ? arr(0)            -> displays array element
' ? name.ToUpper()    -> calls method

' Examples:
' ? a
' 10
' ? a * 5
' 50
' ? name
' "Alice"
' name = "Bob"
```

```
' ? name  
' "Bob"
```

4.6 File I/O (Input/Output)

4.6.1 StreamReader and StreamWriter

StreamWriter – Writing to Files

```
Imports System.IO

Private Sub StreamWriterExamples()
    ' Write text to file (overwrites)
    Dim writer As New StreamWriter("C:\data\output.txt")
    writer.WriteLine("Hello, World!")
    writer.WriteLine("This is line 2")
    writer.Close()

    ' Append to file
    Dim appender As New StreamWriter("C:\data\output.txt", True)
    appender.WriteLine("This line is appended")
    appender.Close()

    ' Using block (auto-closes)
    Using sw As New StreamWriter("C:\data\output.txt")
        sw.WriteLine("Line 1")
        sw.WriteLine("Line 2")
    End Using
End Sub
```

StreamReader – Reading from Files

```
Private Sub StreamReaderExamples()
    ' Read line by line
    Dim reader As New StreamReader("C:\data\input.txt")
    Dim line As String = reader.ReadLine()
    While line IsNot Nothing
        MessageBox.Show(line)
        line = reader.ReadLine()
    End While
    reader.Close()

    ' Read all text at once
    Dim allText As String
```

```

    Using sr As New StreamReader("C:\data\input.txt")
        allText = sr.ReadToEnd()
    End Using
    MessageBox.Show(allText)
End Sub

```

4.6.2 File Class – Static Methods

Method	Description
File.Exists(path)	Checks if file exists
File.Copy(src, dest)	Copies file
File.Move(src, dest)	Moves/renames file
File.Delete(path)	Deletes file
File.CreateText(path)	Creates StreamWriter for new file
File.OpenText(path)	Creates StreamReader for existing file
File.ReadAllText(path)	Reads entire file as string
File.WriteAllText(path, text)	Writes string to file
File.ReadAllLines(path)	Reads all lines as string array
File.WriteAllLines(path, lines)	Writes string array to file

```

Private Sub FileClassExamples()
    Dim path As String = "C:\data\test.txt"

    ' Check existence
    If File.Exists(path) Then
        ' Read all text
        Dim content As String = File.ReadAllText(path)

        ' Read all lines
        Dim lines() As String = File.ReadAllLines(path)

        ' Copy
        File.Copy(path, "C:\data\backup.txt", True)

        ' Move
        File.Move(path, "C:\data\archive\test.txt")
    End If
End Sub

```

```
        ' Delete
        File.Delete("C:\data\backup.txt")
    End If

    ' Write all text
    File.WriteAllText("C:\data\newfile.txt", "Hello, World!")

    ' Write all lines
    Dim data() As String = {"Line 1", "Line 2", "Line 3"}
    File.WriteAllLines("C:\data\lines.txt", data)

    ' Create and open text
    Using writer As StreamWriter = File.CreateText("C:\data\note.txt")
        writer.WriteLine("New file created")
    End Using

    Using reader As StreamReader = File.OpenText("C:\data\note.txt")
        MessageBox.Show(reader.ReadToEnd())
    End Using
End Sub
```

4.6.3 Directory Class – Static Methods

Method	Description
Directory.Exists(path)	Checks if directory exists
Directory.CreateDirectory(path)	Creates directory
Directory.GetFiles(path)	Gets all file paths in directory
Directory.GetDirectories(path)	Gets all subdirectory paths
Directory.Delete(path)	Deletes empty directory
Directory.Move(src, dest)	Moves/renames directory

```
Private Sub DirectoryExamples()
    Dim dir As String = "C:\data\MyFolder"

    ' Create
    If Not Directory.Exists(dir) Then
        Directory.CreateDirectory(dir)
    End If
End Sub
```

```
' Get files
Dim files() As String = Directory.GetFiles(dir, "*.txt")
For Each f As String In files
    MessageBox.Show(Path.GetFileName(f))
Next

' Get subdirectories
Dim subDirs() As String = Directory.GetDirectories(dir)

' Move
Directory.Move(dir, "C:\data\RenamedFolder")

' Delete (must be empty unless recursive)
Directory.Delete("C:\data\OldFolder", True) ' True = recursive
End Sub
```

4.6.4 File Dialogs

OpenFileDialog

```
Private Sub btnOpen_Click(sender As Object, e As EventArgs) Handles btnOpen.Click
    Dim openDlg As New OpenFileDialog()
    openDlg.Title = "Select a file to open"
    openDlg.Filter = "Text Files (*.txt)|*.txt|All Files (*.*)|*.*"
    openDlg.InitialDirectory = "C:\"
    openDlg.Multiselect = False

    If openDlg.ShowDialog() = DialogResult.OK Then
        Dim filePath As String = openDlg.FileName
        Dim content As String = File.ReadAllText(filePath)
        txtContent.Text = content
    End If
End Sub
```

SaveFileDialog

```
Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles btnSave.Click
    Dim saveDlg As New SaveFileDialog()
    saveDlg.Title = "Save file"
    saveDlg.Filter = "Text File (*.txt)|*.txt|Rich Text (*.rtf)|*.rtf"
    saveDlg.DefaultExt = ".txt"
    saveDlg.FileName = "Document1"
```

```
If saveDlg.ShowDialog() = DialogResult.OK Then
    File.WriteAllText(saveDlg.FileName, txtContent.Text)
    MessageBox.Show("File saved successfully!", "Saved",
        MessageBoxButtons.OK, MessageBoxIcon.Information)
End If
End Sub
```

FolderBrowserDialog

```
Private Sub btnBrowse_Click(sender As Object, e As EventArgs) Handles
    btnBrowse.Click
    Dim folderDlg As New FolderBrowserDialog()
    folderDlg.Description = "Select a folder"
    folderDlg.SelectedPath = "C:\"
    folderDlg.ShowNewFolderButton = True

    If folderDlg.ShowDialog() = DialogResult.OK Then
        Dim selectedFolder As String = folderDlg.SelectedPath
        txtFolder.Text = selectedFolder

        ' List files in selected folder
        Dim files() As String = Directory.GetFiles(selectedFolder)
        lstFiles.Items.Clear()
        For Each f As String In files
            lstFiles.Items.Add(Path.GetFileName(f))
        Next
    End If
End Sub
```

Complete File I/O Example

```
Public Class Form1
    Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles
        btnSave.Click
        Using sfd As New SaveFileDialog()
            sfd.Filter = "Text File|*.txt"
            If sfd.ShowDialog() = DialogResult.OK Then
                File.WriteAllText(sfd.FileName, txtEditor.Text)
                MessageBox.Show("File saved.", "Success")
            End If
        End Using
    End Sub
End Class
```

```
Private Sub btnOpen_Click(sender As Object, e As EventArgs) Handles  
btnOpen.Click  
    Using ofd As New OpenFileDialog()  
        ofd.Filter = "Text File|*.txt|All Files|*.*"  
        If ofd.ShowDialog() = DialogResult.OK Then  
            txtEditor.Text = File.ReadAllText(ofd.FileName)  
        End If  
    End Using  
End Sub  
  
Private Sub btnDelete_Click(sender As Object, e As EventArgs) Handles  
btnDelete.Click  
    Using ofd As New OpenFileDialog()  
        If ofd.ShowDialog() = DialogResult.OK Then  
            If File.Exists(ofd.FileName) Then  
                File.Delete(ofd.FileName)  
                MessageBox.Show("File deleted.", "Success")  
            End If  
        End If  
    End Using  
End Sub  
End Class
```

4.7 Database Access with ADO.NET

4.7.1 ADO.NET Architecture

ADO.NET provides data access between .NET applications and databases. Key components:

Component	Description
Connection	Establishes connection to database
Command	Executes SQL queries/stored procedures
DataReader	Forward-only, read-only stream of data
DataAdapter	Bridges DataSet with database (fills DataSet, updates database)
DataSet	In-memory cache of data (disconnected)
DataTable	Represents one table in memory

4.7.2 Connection String

```
' SQL Server connection string
Dim connString As String = "Data Source=.\SQLEXPRESS;Initial
Catalog=CollegeDB;Integrated Security=True"

' SQL Server with username/password
' Dim connString As String = "Server=.;Database=CollegeDB;User
Id=sa;Password=pass123"

' Access database
' Dim connString As String = "Provider=Microsoft.ACE.OLEDB.12.0;Data
Source=C:\data\CollegeDB.accdb"
```

4.7.3 SQL Statements

SQL Statement	Description
SELECT column1, column2 FROM table WHERE condition	Retrieves data
INSERT INTO table (col1, col2) VALUES (val1, val2)	Inserts new record
UPDATE table SET col1 = val1 WHERE condition	Updates existing records

DELETE FROM table WHERE condition	Deletes records
-----------------------------------	-----------------

4.7.4 Using SqlConnection and SqlCommand

```
Imports System.Data.SqlClient
```

```
Public Class Form1
```

```
    Private connectionString As String = "Data Source=.\SQLEXPRESS;Initial  
Catalog=CollegeDB;Integrated Security=True"
```

```
    Private Sub btnConnect_Click(sender As Object, e As EventArgs) Handles  
btnConnect.Click
```

```
        Using conn As New SqlConnection(connectionString)
```

```
            Try
```

```
                conn.Open()
```

```
                MessageBox.Show("Connection successful!", "Success",
```

```
                                MessageBoxButtons.OK, MessageBoxIcon.Information)
```

```
            Catch ex As Exception
```

```
                MessageBox.Show("Error: " & ex.Message, "Connection Failed",
```

```
                                MessageBoxButtons.OK, MessageBoxIcon.Error)
```

```
            End Try
```

```
        End Using
```

```
    End Sub
```

```
    ' SELECT using DataReader
```

```
    Private Sub btnLoad_Click(sender As Object, e As EventArgs) Handles  
btnLoad.Click
```

```
        Dim query As String = "SELECT StudentID, Name, Age FROM Students"
```

```
        Using conn As New SqlConnection(connectionString)
```

```
            Using cmd As New SqlCommand(query, conn)
```

```
                conn.Open()
```

```
                Using reader As SqlDataReader = cmd.ExecuteReader()
```

```
                    lstStudents.Items.Clear()
```

```
                    While reader.Read()
```

```
                        Dim info As String = reader("StudentID").ToString() & " - "
```

```
&
```

```
                        reader("Name").ToString() & " (" &
```

```
                        reader("Age").ToString() & ")"
```

```
                        lstStudents.Items.Add(info)
```

```
                    End While
```

```
                End Using
```

```
            End Using
```

```
        End Using
```

```
End Sub

' INSERT
Private Sub btnInsert_Click(sender As Object, e As EventArgs) Handles
btnInsert.Click
    Dim query As String = "INSERT INTO Students (Name, Age, Course) VALUES
(@name, @age, @course)"

    Using conn As New SqlConnection(connectionString)
        Using cmd As New SqlCommand(query, conn)
            cmd.Parameters.AddWithValue("@name", txtName.Text)
            cmd.Parameters.AddWithValue("@age", CInt(txtAge.Text))
            cmd.Parameters.AddWithValue("@course", cboCourse.Text)

            conn.Open()
            Dim rowsAffected As Integer = cmd.ExecuteNonQuery()
            MessageBox.Show(rowsAffected & " record(s) inserted.", "Success")
        End Using
    End Using
End Sub

' UPDATE
Private Sub btnUpdate_Click(sender As Object, e As EventArgs) Handles
btnUpdate.Click
    Dim query As String = "UPDATE Students SET Age = @age, Course = @course
WHERE StudentID = @id"

    Using conn As New SqlConnection(connectionString)
        Using cmd As New SqlCommand(query, conn)
            cmd.Parameters.AddWithValue("@id", CInt(txtID.Text))
            cmd.Parameters.AddWithValue("@age", CInt(txtAge.Text))
            cmd.Parameters.AddWithValue("@course", cboCourse.Text)

            conn.Open()
            cmd.ExecuteNonQuery()
            MessageBox.Show("Record updated.", "Success")
        End Using
    End Using
End Sub

' DELETE
Private Sub btnDelete_Click(sender As Object, e As EventArgs) Handles
btnDelete.Click
    Dim query As String = "DELETE FROM Students WHERE StudentID = @id"
```

```
        Using conn As New SqlConnection(connectionString)
            Using cmd As New SqlCommand(query, conn)
                cmd.Parameters.AddWithValue("@id", CInt(txtID.Text))

                conn.Open()
                cmd.ExecuteNonQuery()
                MessageBox.Show("Record deleted.", "Success")
            End Using
        End Using
    End Sub
End Class
```

4.7.5 Using DataAdapter and DataSet

```
Private Sub LoadUsingDataAdapter()
    Dim query As String = "SELECT * FROM Students"
    Dim ds As New DataSet()

    Using conn As New SqlConnection(connectionString)
        Using da As New SqlDataAdapter(query, conn)
            da.Fill(ds, "Students")
        End Using
    End Using

    ' Access DataTable from DataSet
    Dim dt As DataTable = ds.Tables("Students")

    ' Iterate rows
    For Each row As DataRow In dt.Rows
        MessageBox.Show(row("Name").ToString())
    Next

    ' Bind to DataGridView
    DataGridView1.DataSource = dt
End Sub
```

4.7.6 Using BindingSource and DataGridView

```
Private Sub LoadWithBindingSource()
    Dim query As String = "SELECT StudentID, Name, Age, Course FROM Students"
    Dim dt As New DataTable()

    Using conn As New SqlConnection(connectionString)
```

```
        Using da As New SqlDataAdapter(query, conn)
            da.Fill(dt)
        End Using
    End Using

    ' BindingSource
    Dim bs As New BindingSource()
    bs.DataSource = dt

    ' Bind to DataGridView
    DataGridView1.DataSource = bs

    ' Bind to individual controls (automatic navigation)
    txtID.DataBindings.Add("Text", bs, "StudentID")
    txtName.DataBindings.Add("Text", bs, "Name")
    txtAge.DataBindings.Add("Text", bs, "Age")
    cboCourse.DataBindings.Add("Text", bs, "Course")
End Sub

' Navigating with BindingSource
Private Sub btnFirst_Click(sender As Object, e As EventArgs) Handles btnFirst.Click
    BindingSource1.MoveFirst()
End Sub

Private Sub btnPrevious_Click(sender As Object, e As EventArgs) Handles
    btnPrevious.Click
    BindingSource1.MovePrevious()
End Sub

Private Sub btnNext_Click(sender As Object, e As EventArgs) Handles btnNext.Click
    BindingSource1.MoveNext()
End Sub

Private Sub btnLast_Click(sender As Object, e As EventArgs) Handles btnLast.Click
    BindingSource1.MoveLast()
End Sub

' Save changes back to database
Private Sub btnSaveChanges_Click(sender As Object, e As EventArgs) Handles
    btnSaveChanges.Click
    Dim dt As DataTable = CType(BindingSource1.DataSource, DataTable)
    Dim query As String = "SELECT StudentID, Name, Age, Course FROM Students"

    Using conn As New SqlConnection(connectionString)
        Using da As New SqlDataAdapter(query, conn)
```

```

        Dim builder As New SqlCommandBuilder(da)
        da.Update(dt)
    End Using
End Using

MessageBox.Show("Changes saved to database.", "Success")
End Sub

```

4.7.7 Complete Database Application Example

```

Public Class StudentForm
    Private connString As String = "Data Source=.\SQLEXPRESS;Initial
Catalog=CollegeDB;Integrated Security=True"
    Private dtStudents As DataTable
    Private daStudents As SqlDataAdapter
    Private bsStudents As BindingSource

    Private Sub StudentForm_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
        LoadData()
    End Sub

    Private Sub LoadData()
        dtStudents = New DataTable()
        Using conn As New SqlConnection(connString)
            daStudents = New SqlDataAdapter("SELECT * FROM Students", conn)
            daStudents.Fill(dtStudents)
        End Using

        bsStudents = New BindingSource()
        bsStudents.DataSource = dtStudents
        DataGridView1.DataSource = bsStudents

        ' Bind controls
        txtID.DataBindings.Add("Text", bsStudents, "StudentID")
        txtName.DataBindings.Add("Text", bsStudents, "Name")
        txtAge.DataBindings.Add("Text", bsStudents, "Age")
    End Sub

    Private Sub btnAdd_Click(sender As Object, e As EventArgs) Handles btnAdd.Click
        bsStudents.AddNew()
    End Sub

    Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles

```

```
btnSave.Click
    bsStudents.EndEdit()
    Using conn As New SqlConnection(connString)
        Dim builder As New SqlCommandBuilder(daStudents)
        daStudents.Update(dtStudents)
    End Using
    MessageBox.Show("Saved successfully.", "Success")
End Sub

Private Sub btnDelete_Click(sender As Object, e As EventArgs) Handles
btnDelete.Click
    If MessageBox.Show("Delete this record?", "Confirm",
        MessageBoxButtons.YesNo, MessageBoxIcon.Question) =
DialogResult.Yes Then
        bsStudents.RemoveCurrent()
        Using conn As New SqlConnection(connString)
            Dim builder As New SqlCommandBuilder(daStudents)
            daStudents.Update(dtStudents)
        End Using
    End If
End Sub
End Class
```

Questions – Unit 4

Section A: Arrays

- Q:** What is an array? Explain the different types of arrays in VB.NET.
-
- Q:** How do you declare and initialize a one-dimensional array? Give examples.
-
- Q:** Write a program to find the sum and average of elements in an integer array.
-
- Q:** Explain the difference between For and For Each loops when working with arrays.
-
- Q:** What is a rectangular array? How is it different from a jagged array?
-
- Q:** Write a program to add two matrices using a 2D array.
-
- Q:** Explain the following Array class methods: Sort, Reverse, Find, IndexOf, Clear, CopyTo, Clone, Resize.
-
- Q:** What is the difference between Length, Rank, and GetLength for arrays?
-
- Q:** Explain how to pass arrays as parameters to procedures with examples.
-
- Q:** Write a program to demonstrate shallow copy vs deep copy of arrays.
-

Section B: Collections

- Q:** What is a collection? Explain the difference between generic and non-generic collections.
-
- Q:** Explain the List(Of T) class with its important methods (Add, Remove, Insert, Sort, Find, Contains).
-
- Q:** What is a Dictionary(Of K, V)? Write a program to store and retrieve employee data.
-
- Q:** Explain the Stack(Of T) class with Push, Pop, Peek methods and LIFO behavior.
-
- Q:** Explain the Queue(Of T) class with Enqueue, Dequeue, Peek methods and FIFO behavior.
-
- Q:** What is an ArrayList? How is it different from List(Of T)?
-

Q: Explain Hashtable and SortedList classes with examples.

Q: Write a program to demonstrate the use of Contains and Count methods in collections.

Q: Differentiate between Stack and Queue with real-world analogies.

Section C: File I/O

Q: Explain the difference between StreamReader and StreamWriter classes.

Q: Write a program to read a text file using StreamReader (line by line and all at once).

Q: Write a program to write and append data to a text file using StreamWriter.

Q: Explain the static methods of the File class: Exists, Copy, Move, Delete, ReadAllText, WriteAllText.

Q: What is the Directory class? Explain CreateDirectory, GetFiles, GetDirectories, Delete, Move.

Q: How do you use OpenFileDialog and SaveFileDialog in a VB.NET application?

Q: Write a program to create a simple text editor with Open, Save, and Delete functionality.

Q: What is the FolderBrowserDialog? Write an example to select a folder and list its files.

Q: Explain the Using block and its importance in file operations.

Q: Write a program to copy a file from one location to another.

Section D: Database Access

Q: What is ADO.NET? Explain its architecture (Connection, Command, DataReader, DataAdapter, DataSet).

Q: How does SqlDataReader work? How is it different from DataAdapter?

Q: Explain the difference between DataSet and DataTable.

Q: Write a program to connect to a SQL Server database and retrieve records using SqlDataReader.

Q: Write the SQL statements for SELECT, INSERT, UPDATE, and DELETE with parameterized queries.

Q: What is a connection string? Write examples for SQL Server and Access databases.

Q: Explain the BindingSource component. How does it simplify data binding?

Q: How do you use DataGridView to display database records?

Q: Write a program to perform CRUD operations (Create, Read, Update, Delete) on a Students table.

Q: What is SqlCommandBuilder? How is it used with DataAdapter to update data?

Q: Explain the importance of the Using block in database connections.

Q: Write a program that uses BindingSource to navigate records (First, Previous, Next, Last).

Section E: Controls, Multi-Form, Debugging

Q: Explain the ComboBox control. How do you add items and retrieve the selected item?

Q: What is the difference between ComboBox and ListBox?

Q: How do CheckBox and RadioButton differ? How does GroupBox affect RadioButton behavior?

Q: How do you set the tab order of controls on a form?

Q: Explain how to add a new form to a project and set it as the startup form.

Q: What is the difference between Show() and ShowDialog() methods?

Q: How do you pass data between forms using properties and constructors?

Q: Explain the FormClosing event. How can you cancel form closure?

Q: What is a breakpoint? Explain conditional breakpoints and tracepoints.

Q: Explain the purpose of Locals, Watch, Call Stack, and Immediate debugging windows.

Q: What is Edit and Continue? How does it help during debugging?

Q: Write a program demonstrating Debug.WriteLine and Trace.WriteLine.
